Open in Colab

Open in Colab

# Lab 8.2 Machine Learning 2 – Artificial Neural Network, Classification

In [1]:
```
# Let's check the installed tensorflow version
import tensorflow as tf

print('TensorFlow Version is', tf.__version__)
```

```
TensorFlow Version is 2.19.0
```

In [2]:
```
# required Python packages for this colab
!pip install matplotlib
!pip install pandas
!pip install scipy
!pip install scikit-learn
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.1
2.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(1.5.0)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.
2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.
0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotl
ib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7
->matplotlib) (1.17.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from panda
s) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.
3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.
8.2->pandas) (1.17.0)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (1.16.3)
Requirement already satisfied: numpy<2.6,>=1.25.2 in /usr/local/lib/python3.12/dist-packages (from scipy) (2.
0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn)
(2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn)
(1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn)
(1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-l
earn) (3.6.0)
```

# Training ML model for anomaly detection

Now, you are ready to train ML model. Follow the steps below to create your autoencoder model to predict the anomalous condition of the AFF. In this lab manual, we will use this Colab Notebook. But, you can use this Colab Notefile (ipynb) on your computer using Jupyter Notebook or creating a Python script.

We will reuse the autoencoder training code in Prelab8.2.

**When you first train the model, please use the given axis, feature, and hyper -parameters to perform Task 2.2. And then, try different combinations.**

## Import TensorFlow and relevant Python packages on your Colab session

In [3]:
```
# Let's check the installed tensorflow version
import tensorflow as tf

print('TensorFlow Version is', tf.__version__)
```

```
TensorFlow Version is 2.19.0
```

In [4]:
```
import os
import matplotlib.pyplot as plt
import datetime
import numpy as np
import pandas as pd
from tensorflow import keras
from scipy import stats, fft

from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_curve, auc
from sklearn.model_selection import train_test_split
from tensorflow.keras import layers, losses
from tensorflow.keras.models import Model
```

## Load data

First, upload your training data (Normal and Abnormal) which you collected on the last part to this Colab session.

- Method 1: Upload file on this Colab session (Fast)
- Method 2: Use the code block below (Slow)
- Method 3: Mount your Google Drive if your data is in it (Fast)

In [5]:
```
#This code allows you to upload a file from your local drive
from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving 20260328_163238_Normal_lab8_data.csv to 20260328_163238_Normal_lab8_data (1).csv
Saving 20260328_165656_Abnormal_lab8_data.csv to 20260328_165656_Abnormal_lab8_data (1).csv
```

In [6]:
```
## Loading data using pandas

normal_data_file = "20260328_163238_Normal_lab8_data.csv" # normal condition filename: You must change this!
abnormal_data_file = "20260328_165656_Abnormal_lab8_data.csv" # abnormal condition filename: You must change

df_normal = pd.read_csv(normal_data_file) # normal dataframe
df_abnormal = pd.read_csv(abnormal_data_file) # abnormal dataframe

frames = [df_normal, df_abnormal] # frame list to merge two dataframes into one

df = pd.concat(frames) # new concatenated dataframe

df # display dataframe
```

Out[6]:

| | Condition | Xacc array [m/s2] | Yacc array [m/s2] | Zacc array [m/s2] |
|---|---|---|---|---|
| **0** | Normal | 0.0 0.2353596 0.0 -0.6276256 0.0 0.0 0.0 0.549... | -0.4707192 1.0983448 -0.2353596 0.0 0.2353596 ... | 10.591182 8.7867584 11.6110736 10.591182 9.492... |
| **1** | Normal | 0.2353596 -0.6276256 -0.4707192 0.5491724 -0.5... | 0.0784532 1.0198916 -0.392266 -0.1569064 0.078... | 9.1790244 10.0420096 8.7083052 10.8265416 9.25... |
| **2** | Normal | 0.2353596 0.0 -0.2353596 0.6276256 -0.5491724 ... | 0.9414384 0.5491724 -0.0784532 -1.2552512 0.0 ... | 9.492837199999999 9.9635564 11.2972608 9.02211... |
| **3** | Normal | 0.0 0.392266 -0.0784532 0.0 0.392266 0.0 0.235... | -0.1569064 0.0784532 -0.8629851999999999 0.078... | 8.629852 8.7867584 9.649743599999999 9.9635564... |
| **4** | Normal | 0.784532 0.4707192 0.0 0.0 0.0 0.0 0.0 -0.3922... | -0.6276256 0.3138128 -0.392266 0.6276256 1.176... | 10.4342756 8.8652116 11.924886399999998 9.1790... |
| **...** | ... | ... | ... | ... |
| **295** | Abnormal | 0.0 0.2353596 0.5491724 -0.7060788 0.2353596 2... | 3.4519407999999996 4.6287388 6.4331624 3.13812... | 11.2188076 10.8265416 9.022118 10.277369199999... |
| **296** | Abnormal | 0.0 -1.1767979999999998 -1.3337044 -1.3337044 ... | 1.4121576 2.9812215999999996 3.373487600000000... | 7.609960399999999 6.354709199999999 9.1790244 ... |
| **297** | Abnormal | -0.3138128 -0.3138128 -0.784532 -1.2552512 -1.... | -0.8629851999999999 0.0 -0.5491724 1.2552512 4... | 7.139241199999999 9.3359308 9.2574776 11.21880... |
| **298** | Abnormal | -0.6276256 -1.8828768 -2.2751428 -1.2552512 -0... | 3.138128 2.9812215999999996 2.5105024 4.158019... | 9.1790244 11.1403544 9.7281968 8.3944923999999... |
| **299** | Abnormal | -1.569064 -1.8828768 -1.3337044 0.2353596 0.23... | 4.8640984 5.1779112 2.9027684 4.314926 1.64751... | 6.982334799999999 6.982334799999999 9.8851032 ... |

600 rows × 4 columns

## Data Transformation

The strategy to build this model is to select, one by one, data from each axis and access which axis helps the model classify with better accuracy and precision.

Let's start with the X-axis: Assign the X-axis on the variable AXIS on the code below.

In [7]:
```
## Data Transformation
# X-axis: 'Xacc array [m/s2]'
# Y-axis: 'Yacc array [m/s2]'
# Z-axis: 'Zacc array [m/s2]'
AXIS = 'Xacc array [m/s2]' # in this case, X-axis acceleration data

# Exploding the values contained in selected column and converting the string values into float values
df_new = pd.concat([df['Condition'],df[AXIS].str.split(' ', expand=True).astype(float)], axis=1) # transform
ds = df_new.copy() # make ds by copying df
```

Now, we change the labels 'Normal' and 'Abnormal' into binary notation for the model usage.

In [8]:
```
#Converting the Classifier into binary values
ds.loc[df['Condition'] == 'Normal', 'Status'] = 1 # if Condition column is 'Normal', Give 'Status' Column 1
ds.loc[df['Condition'] == 'Abnormal', 'Status'] = 0 # if Condition column is 'Abnormal', Give 'Status' Colum
ds.drop('Condition', axis=1, inplace=True) # drop 'Condition' column (the first column)

ds # display ds (dataset)
```

Out[8]:

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | ... | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0.000000 | 0.235360 | 0.000000 | -0.627626 | 0.000000 | 0.000000 | 0.000000 | 0.549172 | 0.156906 | 0.784532 | ... | 0 |
| **1** | 0.235360 | -0.627626 | -0.470719 | 0.549172 | -0.549172 | 0.000000 | -0.392266 | -0.706079 | 0.000000 | 0.000000 | ... | -0 |
| **2** | 0.235360 | 0.000000 | -0.235360 | 0.627626 | -0.549172 | 0.313813 | 0.078453 | -0.235360 | 0.156906 | -0.313813 | ... | 0 |
| **3** | 0.000000 | 0.392266 | -0.078453 | 0.000000 | 0.392266 | 0.000000 | 0.235360 | -0.470719 | -0.156906 | 0.784532 | ... | 0 |
| **4** | 0.784532 | 0.470719 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | -0.392266 | 0.470719 | -0.549172 | ... | -0 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | |
| **295** | 0.000000 | 0.235360 | 0.549172 | -0.706079 | 0.235360 | 2.039783 | 2.039783 | -0.862985 | -1.019892 | -0.941438 | ... | 1 |
| **296** | 0.000000 | -1.176798 | -1.333704 | -1.333704 | -0.784532 | -0.706079 | 0.235360 | 0.000000 | 0.941438 | 1.019892 | ... | 0 |
| **297** | -0.313813 | -0.313813 | -0.784532 | -1.255251 | -1.098345 | -1.882877 | -1.255251 | 0.000000 | -0.549172 | 0.784532 | ... | -0 |
| **298** | -0.627626 | -1.882877 | -2.275143 | -1.255251 | -0.235360 | 0.000000 | 0.235360 | 1.490611 | 1.176798 | -0.235360 | ... | -0 |
| **299** | -1.569064 | -1.882877 | -1.333704 | 0.235360 | 0.235360 | 0.078453 | 0.392266 | -0.078453 | -0.392266 | 0.392266 | ... | 0 |

600 rows × 1001 columns

## Training data pipe-line

We just loaded data and transformed it. Let's start creating input data pipe-line for training the model.

Define data, raw data and labels.

In [9]:
```
data = ds.values
# Define Raw data W/O signal processing
raw_data = data[:,:-1]
# Labels: The last column
labels = data[:,-1]
```

Here is the signal processing part. Time domain and frequency domain will be used for training the model.

Using the signal processing, you can define the time domain feature and frequency domain feature, respectively.

In [10]:
```
## signal processing
# Time-domain data
def timeFeatures(data):
    feature = [] # initialize feature list
    for i in range(len(data)):
        mean = np.mean(data[i]) # mean
        std = np.std(data[i]) # standard deviation
        rms = np.sqrt(np.mean(data[i] ** 2)) # root mean squre
        peak = np.max(abs(data[i])) # peak
        skew = stats.skew(data[i]) # skewness
        kurt = stats.kurtosis(data[i]) # kurtosis
        cf = peak/rms # crest factor
        # number of feature of each measurement = 7
        feature.append(np.array([mean,std,rms,peak,skew,kurt,cf], dtype=float))
    feature = np.array(feature)
    return feature # feature list, each element is numpy array with datatype float

# DFT magnitude data
def freqFeatures(data):
    feature = []
    for i in range(len(data)):
        N = len(data[i]) # number of data
        yf = 2/N*np.abs(fft.fft(data[i])[:N//2]) # yf is DFT signal magnitude
        yf[0] = 0
        feature.append(np.array(yf))
```

```
    feature = np.array(feature)
    return feature


time_data = timeFeatures(raw_data) # define time domain feature
freq_data = freqFeatures(raw_data) # define frequency domain feature (DFT)
```

Here is data (feature) selection and split part.

In [11]:
```
## Data (feature) selection and Split training and validation dataset
# Feature selection
input_feature = raw_data # raw data input without any signal processing
# input_feature = time_data # time domain data
# input_feature = freq_data # frequency data

# get training and test (validation) dataset
train_data, test_data, train_labels, test_labels = train_test_split(
    input_feature, labels, test_size=0.2, random_state=20
)
```

Normalization based on a min/max scaling is done here to support the learning objective of the autoencoder (minimizing reconstruction error) and makes the results more interpretable.

In [12]:
```
## Data normalization

def tensorNormalization(data): # data as numpy array
    min_val = tf.reduce_min(data) # get min val
    max_val = tf.reduce_max(data) # get max val
    data_normal = (data - min_val) / (max_val - min_val) # get normalized data as numpy array
    return tf.cast(data_normal, tf.float32) # tensorarray, float 32 datatype

train_data = tensorNormalization(train_data) # Normalizing train data
test_data = tensorNormalization(test_data) # Normalizing test data
```

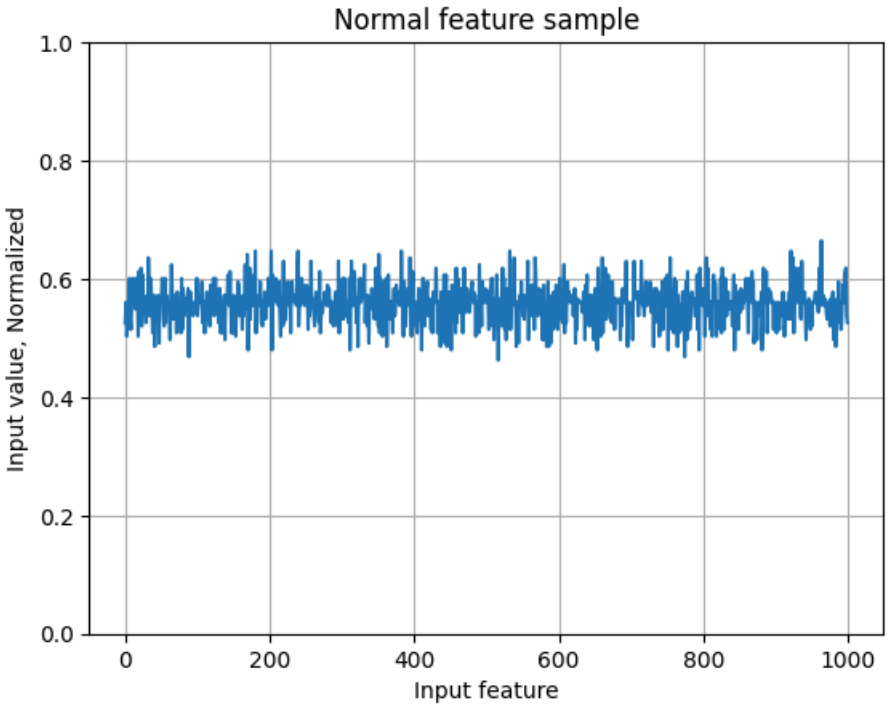
You will train the autoencoder using only the normal rhythms, which are labeled in the dataset as 1. Separate the normal vibrations from the abnormal vibrations. **You have to make sure of the size/dimension of feature array.**

In [13]:
```
#Splitting the dataset based on classification: train_labels: Normal, ~train_labels: Abnormal
train_labels = train_labels.astype(bool) # transform labels as bool (True or False)
test_labels = test_labels.astype(bool) # in case of 'Normal' it is Ture, if not, False

normal_train_data = train_data[train_labels] # define normal train data
normal_test_data = test_data[test_labels] # define normal test data

anomalous_train_data = train_data[~train_labels] # define abnormal train data
anomalous_test_data = test_data[~test_labels] # define abnormal test data

portion_of_anomaly_in_training = 0.1 #10% of training data will be anomalies
end_size = int(len(normal_train_data)/(10-portion_of_anomaly_in_training*10))
combined_train_data = np.append(normal_train_data, anomalous_test_data[:end_size], axis=0)
N_feature = combined_train_data.shape[1] # get the Number of feature, it depends on your input feature!
print("Number of training dataset is {}.".format(combined_train_data.shape))
print("Number of feature is {}.".format(N_feature))
```

```
Number of training dataset is (260, 1000).
Number of feature is 1000.
```

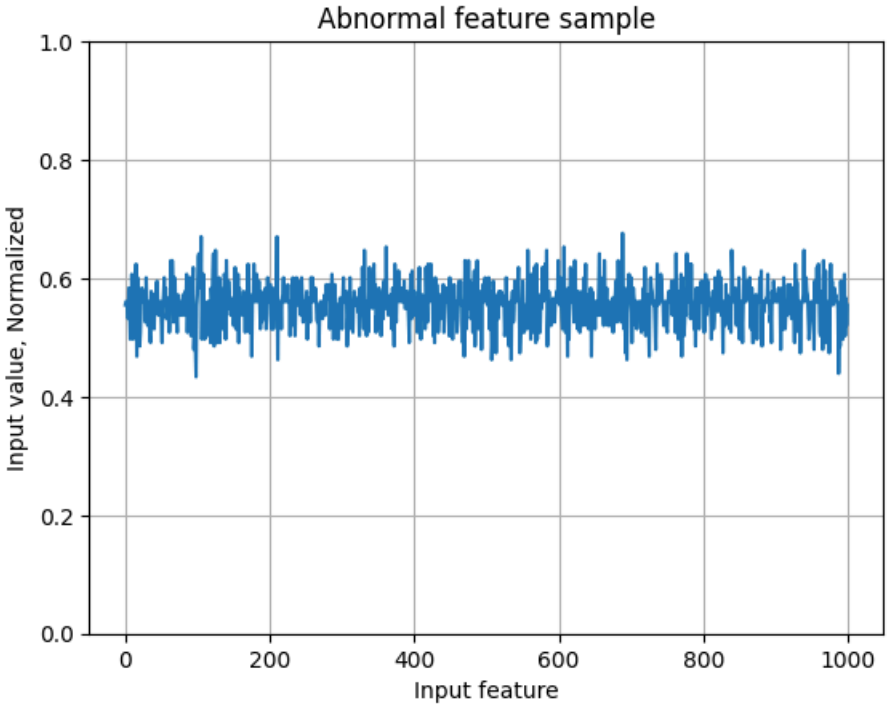
Plot a normal vibration feature

In [14]:
```
#Plotting sample of normal data
plt.grid()
plt.plot(np.arange(N_feature), normal_train_data[0])
plt.title("Normal feature sample")
plt.ylim([0, 1])
plt.xlabel("Input feature")
plt.ylabel("Input value, Normalized")
plt.show()
```




Plot a abnomalous vibration feature

In [15]:
```
#Plotting sample of anomalous data
plt.grid()
plt.plot(np.arange(N_feature), anomalous_train_data[0])
plt.title("Abnormal feature sample")
plt.ylim([0, 1])
plt.xlabel("Input feature")
plt.ylabel("Input value, Normalized")
plt.show()
```




## Task 2.1

What differences do you notice between the normal and anomalous plots? Can you figure out anomaly based on the siganl/feature?

---

The abnormal data seems to have a higher amplitude and a faster amplitude than the normal data. However the graphs are very similar since this is based on the X-axis data and likely would change if a different axis was used.

---

## Autoencoder model training

Changing the size of the embedding (latent space) can produce interesting results. Try to play around with that layer size by assigning a value to the variable EMBEDDING_SIZE.

In [16]:
```
## Creating the artificial neural network using autoencoder
EMBEDDING_SIZE = 64 #Define how many neurons in the inner layer   <----------------------------------------
class AnomalyDetector(Model):
  def __init__(self):
    super(AnomalyDetector, self).__init__()
    self.encoder = tf.keras.Sequential([
      layers.Dense(256, activation="relu"),
      layers.Dense(128, activation="relu"),
      layers.Dense(EMBEDDING_SIZE, activation="relu")]) # Smallest Layer Defined Here

    self.decoder = tf.keras.Sequential([
      layers.Dense(128, activation="relu"),
      layers.Dense(256, activation="relu"),
      layers.Dense(N_feature, activation="sigmoid")])

  def call(self, x):
    encoded = self.encoder(x)
    decoded = self.decoder(encoded)
    return decoded

autoencoder = AnomalyDetector() # define autoencoder model class
print("Chosen Embedding Size: ", EMBEDDING_SIZE)
# opt = keras.optimizers.Adam(learning_rate=0.0001) # default learning rate is 0.001, if you want, you can c
# autoencoder.compile(optimizer=opt, loss='mae')
autoencoder.compile(optimizer='adam', loss='mae')
```

```
Chosen Embedding Size:  64
```

## Training the model

Note that the autoencoder is trained using only the normal data, but is evaluated using the full test set.

In [17]:
```
#Training the model
history = autoencoder.fit(normal_train_data, normal_train_data,
          epochs=100,
          batch_size=100,
          validation_data=(test_data, test_data),
          shuffle=True)
```

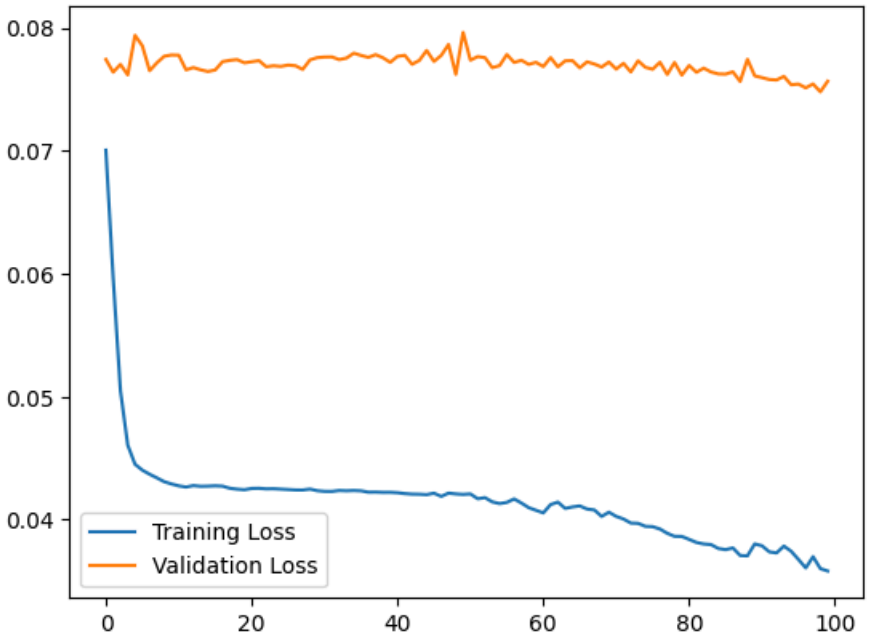
```
Epoch 1/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 2s 126ms/step - loss: 0.0700 - val_loss: 0.0774
Epoch 2/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0597 - val_loss: 0.0764
Epoch 3/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0505 - val_loss: 0.0770
Epoch 4/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0461 - val_loss: 0.0762
Epoch 5/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0445 - val_loss: 0.0794
Epoch 6/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 52ms/step - loss: 0.0440 - val_loss: 0.0785
Epoch 7/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0437 - val_loss: 0.0765
Epoch 8/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0434 - val_loss: 0.0771
Epoch 9/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0431 - val_loss: 0.0777
Epoch 10/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0429 - val_loss: 0.0778
Epoch 11/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0427 - val_loss: 0.0778
Epoch 12/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0426 - val_loss: 0.0766
Epoch 13/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0428 - val_loss: 0.0768
Epoch 14/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step - loss: 0.0427 - val_loss: 0.0766
Epoch 15/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.0427 - val_loss: 0.0764
Epoch 16/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0427 - val_loss: 0.0766
Epoch 17/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.0427 - val_loss: 0.0772
Epoch 18/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0425 - val_loss: 0.0773
Epoch 19/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0425 - val_loss: 0.0774
Epoch 20/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0424 - val_loss: 0.0771
Epoch 21/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.0425 - val_loss: 0.0772
Epoch 22/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step - loss: 0.0425 - val_loss: 0.0773
Epoch 23/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.0425 - val_loss: 0.0768
Epoch 24/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0425 - val_loss: 0.0769
Epoch 25/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.0425 - val_loss: 0.0769
Epoch 26/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0424 - val_loss: 0.0770
Epoch 27/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0424 - val_loss: 0.0769
Epoch 28/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.0424 - val_loss: 0.0766
Epoch 29/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 86ms/step - loss: 0.0425 - val_loss: 0.0774
Epoch 30/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 81ms/step - loss: 0.0424 - val_loss: 0.0776
Epoch 31/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 79ms/step - loss: 0.0423 - val_loss: 0.0776
Epoch 32/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 64ms/step - loss: 0.0423 - val_loss: 0.0776
Epoch 33/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 77ms/step - loss: 0.0424 - val_loss: 0.0774
Epoch 34/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 87ms/step - loss: 0.0423 - val_loss: 0.0775
Epoch 35/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 68ms/step - loss: 0.0424 - val_loss: 0.0779
```

```
Epoch 36/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 69ms/step - loss: 0.0423 - val_loss: 0.0777
Epoch 37/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 72ms/step - loss: 0.0422 - val_loss: 0.0776
Epoch 38/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 69ms/step - loss: 0.0422 - val_loss: 0.0778
Epoch 39/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 92ms/step - loss: 0.0422 - val_loss: 0.0776
Epoch 40/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 76ms/step - loss: 0.0422 - val_loss: 0.0772
Epoch 41/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 75ms/step - loss: 0.0422 - val_loss: 0.0777
Epoch 42/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.0421 - val_loss: 0.0778
Epoch 43/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0421 - val_loss: 0.0770
Epoch 44/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0421 - val_loss: 0.0773
Epoch 45/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 52ms/step - loss: 0.0420 - val_loss: 0.0781
Epoch 46/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0421 - val_loss: 0.0773
Epoch 47/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.0419 - val_loss: 0.0777
Epoch 48/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0422 - val_loss: 0.0786
Epoch 49/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0421 - val_loss: 0.0762
Epoch 50/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0420 - val_loss: 0.0796
Epoch 51/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0421 - val_loss: 0.0774
Epoch 52/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0417 - val_loss: 0.0776
Epoch 53/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step - loss: 0.0418 - val_loss: 0.0776
Epoch 54/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0414 - val_loss: 0.0768
Epoch 55/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0413 - val_loss: 0.0769
Epoch 56/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.0414 - val_loss: 0.0778
Epoch 57/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0417 - val_loss: 0.0772
Epoch 58/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0413 - val_loss: 0.0773
Epoch 59/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0410 - val_loss: 0.0770
Epoch 60/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step - loss: 0.0408 - val_loss: 0.0772
Epoch 61/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.0405 - val_loss: 0.0768
Epoch 62/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0412 - val_loss: 0.0776
Epoch 63/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0414 - val_loss: 0.0768
Epoch 64/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0409 - val_loss: 0.0773
Epoch 65/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0410 - val_loss: 0.0773
Epoch 66/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0411 - val_loss: 0.0767
Epoch 67/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0409 - val_loss: 0.0772
Epoch 68/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step - loss: 0.0408 - val_loss: 0.0770
Epoch 69/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step - loss: 0.0403 - val_loss: 0.0768
Epoch 70/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0406 - val_loss: 0.0772
```

```
Epoch 71/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0403 - val_loss: 0.0766
Epoch 72/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0400 - val_loss: 0.0771
Epoch 73/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0397 - val_loss: 0.0764
Epoch 74/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.0397 - val_loss: 0.0773
Epoch 75/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step - loss: 0.0395 - val_loss: 0.0768
Epoch 76/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step - loss: 0.0394 - val_loss: 0.0766
Epoch 77/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0392 - val_loss: 0.0772
Epoch 78/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0389 - val_loss: 0.0762
Epoch 79/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0386 - val_loss: 0.0772
Epoch 80/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0386 - val_loss: 0.0761
Epoch 81/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0384 - val_loss: 0.0769
Epoch 82/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step - loss: 0.0381 - val_loss: 0.0764
Epoch 83/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step - loss: 0.0380 - val_loss: 0.0767
Epoch 84/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.0380 - val_loss: 0.0764
Epoch 85/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0377 - val_loss: 0.0762
Epoch 86/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0376 - val_loss: 0.0762
Epoch 87/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0377 - val_loss: 0.0764
Epoch 88/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0371 - val_loss: 0.0756
Epoch 89/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0371 - val_loss: 0.0774
Epoch 90/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0380 - val_loss: 0.0761
Epoch 91/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step - loss: 0.0379 - val_loss: 0.0759
Epoch 92/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0374 - val_loss: 0.0758
Epoch 93/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0373 - val_loss: 0.0758
Epoch 94/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0378 - val_loss: 0.0760
Epoch 95/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step - loss: 0.0374 - val_loss: 0.0754
Epoch 96/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.0368 - val_loss: 0.0754
Epoch 97/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0361 - val_loss: 0.0751
Epoch 98/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 52ms/step - loss: 0.0370 - val_loss: 0.0754
Epoch 99/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step - loss: 0.0360 - val_loss: 0.0748
Epoch 100/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0358 - val_loss: 0.0756
```

In [18]:
```
#Plotting the evolution of training and validation loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.legend()
```

Out[18]:
```
<matplotlib.legend.Legend at 0x79583d1bb830>
```

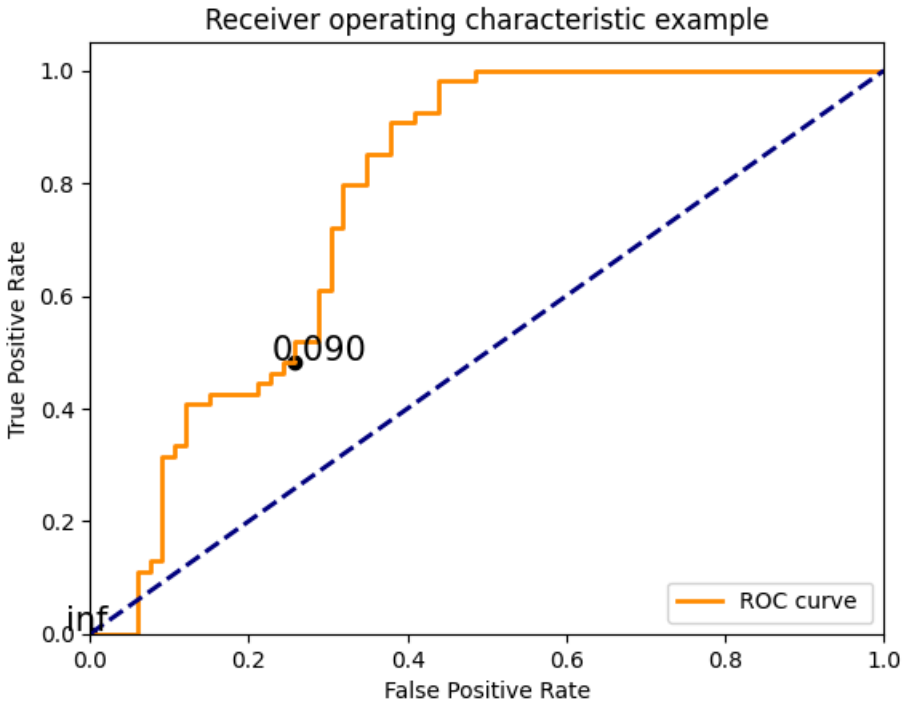



## Evaluation and determination of threshold

The Receiver Operating Characteristic (ROC) plots allows us to visualize the tradeoff between predicting anomalies as normal (false positives) and predicting normal data as an anomaly (false negative). Normal vibrations are labeled as 1 in this dataset but we have to flip them here to match the ROC curves expectations.

The ROC plot now has threshold values plotted on their corresponding points on the curve to aid in selecting a theshold for the application.

In [19]:
```
#Plotting True positive and false positive rate assessment
reconstructions = autoencoder(test_data)
loss = tf.keras.losses.mae(reconstructions, test_data)
fpr = []
tpr = []
#the test labels are flipped to match how the roc_curve function expects them.
# flipped_labels = 1-test_labels
flipped_labels = 1-test_labels
fpr, tpr, thresholds = roc_curve(flipped_labels, loss)
plt.figure()
lw = 2
plt.plot(fpr, tpr, color='darkorange',
         lw=lw, label='ROC curve ')
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic example')
plt.legend(loc="lower right")

# plot some thresholds
thresholds_every=20
thresholdsLength = len(thresholds)
colorMap=plt.get_cmap('jet', thresholdsLength)
for i in range(0, thresholdsLength, thresholds_every):
  threshold_value_with_max_four_decimals = str(thresholds[i])[:5]
  plt.scatter(fpr[i], tpr[i], c='black')
  plt.text(fpr[i] - 0.03, tpr[i] + 0.005, threshold_value_with_max_four_decimals, fontdict={'size': 15})

plt.show()
```

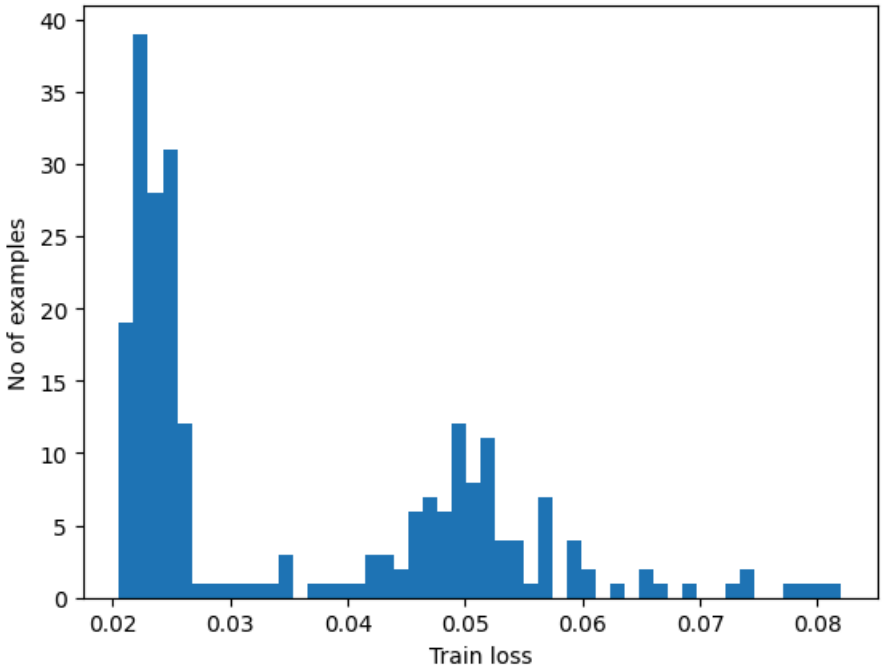



In [20]:
```
roc_auc = auc(fpr, tpr) # check roc auc score (the area of roc graph above)
print("ROC - AUC score is {}.".format(roc_auc))
```

```
ROC - AUC score is 0.7738496071829405.
```

In [21]:
```
## reconstructions and histogram of training model
reconstructions = autoencoder.predict(normal_train_data)
train_loss = tf.keras.losses.mae(reconstructions, normal_train_data)

plt.hist(train_loss[None,:], bins=50)
plt.xlabel("Train loss")
plt.ylabel("No of examples")
plt.show()
```

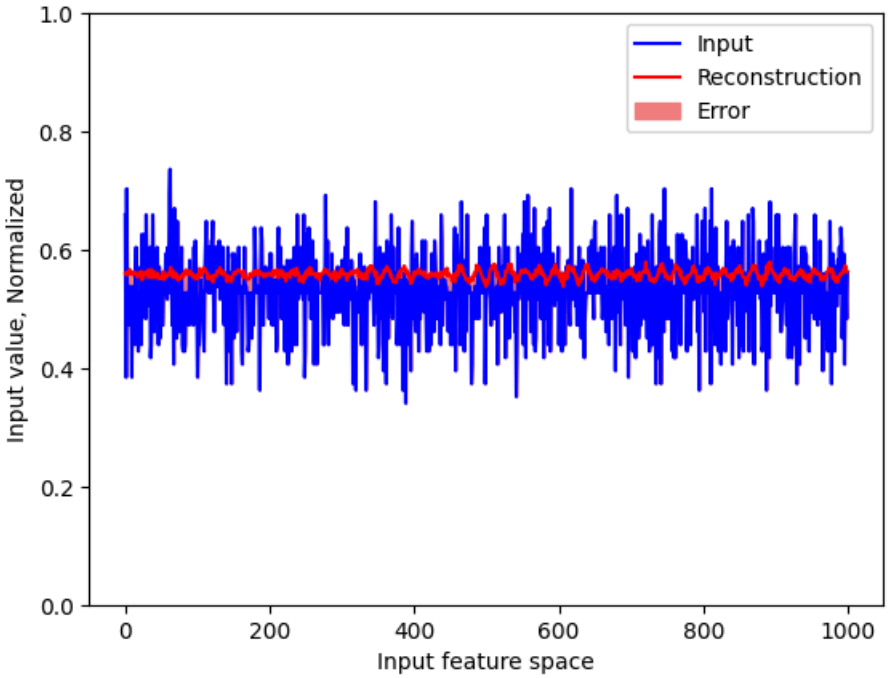
```
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step
```



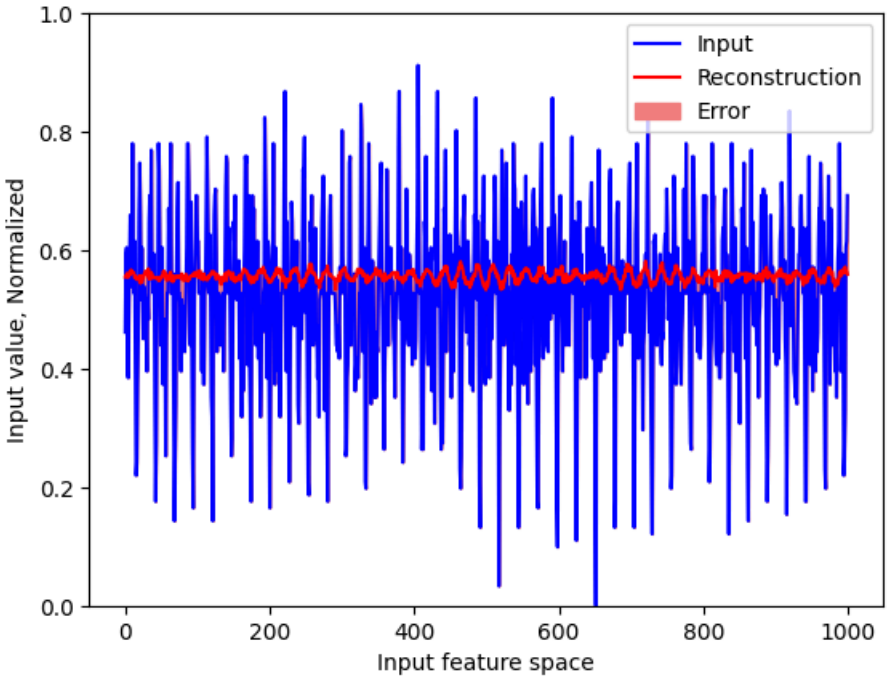
In [99]:
```
## Determining the threshold to detect anomalies
# threshold =  #If you want to assign a value labeled in black in the ROC graph  <--------------------------
threshold = np.mean(train_loss) + np.std(train_loss) # Common rule of thumb based on the histogram of the tr
print("Threshold: ", threshold) # You have to note the threshold you will use

def predict(model, data, threshold):
  reconstructions = model(data)
  loss = tf.keras.losses.mae(reconstructions, data)
  # For anomaly detection, 'True' typically means anomalous. If loss >= threshold, it's predicted as anomalo
  return tf.math.greater_equal(loss, threshold), loss

def print_stats(predictions, labels):
  print("Accuracy = {}".format(accuracy_score(labels, predictions)))
  print("Precision = {}".format(precision_score(labels, predictions)))
  print("Recall = {}".format(recall_score(labels, predictions)))

# Re-evaluate flipped_labels here for consistency with ROC curve calculation
flipped_labels = 1 - best_data.test_labels

preds, scores = predict(best_model, best_data.test_data, threshold)
# Now, preds is True for predicted ANOMALOUS, False for predicted NORMAL
# flipped_labels is True for actual ANOMALOUS, False for actual NORMAL
print_stats(preds, flipped_labels)
```

```
Threshold:  0.035335165
Accuracy = 0.45
Precision = 0.45
Recall = 1.0
```

In [23]:
```
# Plot "Normal" test data sample with reconstruction based on created model and error
encoded_data = autoencoder.encoder(normal_test_data).numpy()
decoded_data = autoencoder.decoder(encoded_data).numpy()

plt.plot(normal_test_data[0], 'b')
plt.plot(decoded_data[0], 'r')
plt.fill_between(np.arange(N_feature), decoded_data[0], normal_test_data[0], color='lightcoral')
plt.ylim([0, 1])
plt.legend(labels=["Input", "Reconstruction", "Error"])
plt.xlabel("Input feature space")
plt.ylabel("Input value, Normalized")
plt.show()
```



In [24]:
```
# Plot "Abnormal" test data sample with reconstruction based on created model and error
encoded_data = autoencoder.encoder(anomalous_test_data).numpy()
decoded_data = autoencoder.decoder(encoded_data).numpy()

plt.plot(anomalous_test_data[0], 'b')
plt.plot(decoded_data[0], 'r')
plt.fill_between(np.arange(N_feature), decoded_data[0], anomalous_test_data[0], color='lightcoral')
plt.ylim([0, 1])
plt.legend(labels=["Input", "Reconstruction", "Error"])
plt.xlabel("Input feature space")
plt.ylabel("Input value, Normalized")
plt.show()
```




## Task 2.2

After finishing training the model so far in the condition below, answer the following questions.

- Acceleration axis: X-axis
- input feature: Raw data (without signal processing as Prelab8)
- Embedding size: 64
- Please specify if you change any hyper-parameters or variables.

1. What is performance of the model? Evaluate and analyze your model including Threshold, Accuracy, Precision, Recall, and so on. How did you determine the threshold?

---

Threshold: 0.057505924
Accuracy = 0.6916666666666667
Precision = 0.7377049180327869
Recall = 0.6818181818181818

The performance of the model is not very good. Its accuracy, precision and recall are all below 90% which means it may not be very useful for reconstructing a normal signal.

The threshold was determined was determined using the given rule of thumb based on the histogram of the train loss.

---

2. Do you think your model is good enough for anomaly detection of the AFF?

---

No, unfortunately I do not think that the model is good enough for anomoly detection of the AFF.

---

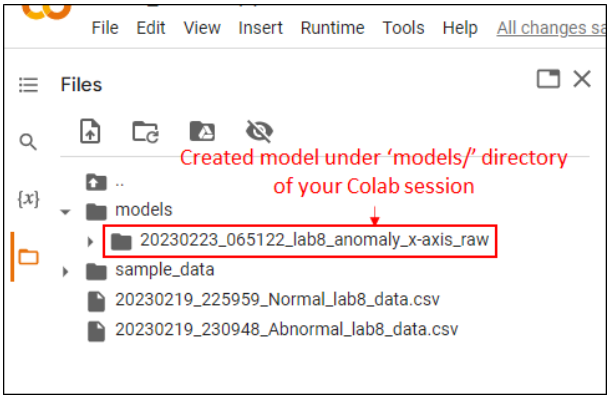
## Save the model and reload it

After you train a model, you should save the model for implementation in the future.

In [25]:
```
# create the "models" folder first
model_folder = "models"
if not os.path.exists(model_folder):
    os.mkdir(model_folder)

t = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
model_identifier = "_lab8_anomaly_x-axis_raw"

# SavedModel directory (NO extension)
export_path = "./models/" + t + model_identifier

# build model (keep)
dummy_input = tf.zeros((1, N_feature))
_ = autoencoder(dummy_input)

# export SavedModel directory
autoencoder.export(export_path)

print("SavedModel exported to:", export_path)
```

```
Saved artifact at './models/20260329_222346_lab8_anomaly_x-axis_raw'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 1000), dtype=tf.float32, name=None)
Output Type:
  TensorSpec(shape=(None, 1000), dtype=tf.float32, name=None)
Captures:
  133419889866896: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133419889866320: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133419889865360: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133419888607696: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133419888609040: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133419888608464: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133419888609424: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133419888609232: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133419888609808: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133419888609616: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133419888610192: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133419888610000: TensorSpec(shape=(), dtype=tf.resource, name=None)
SavedModel exported to: ./models/20260329_222346_lab8_anomaly_x-axis_raw
```

After you save the model, you see the created model directory under 'models/' in your Colab session as in the example below.



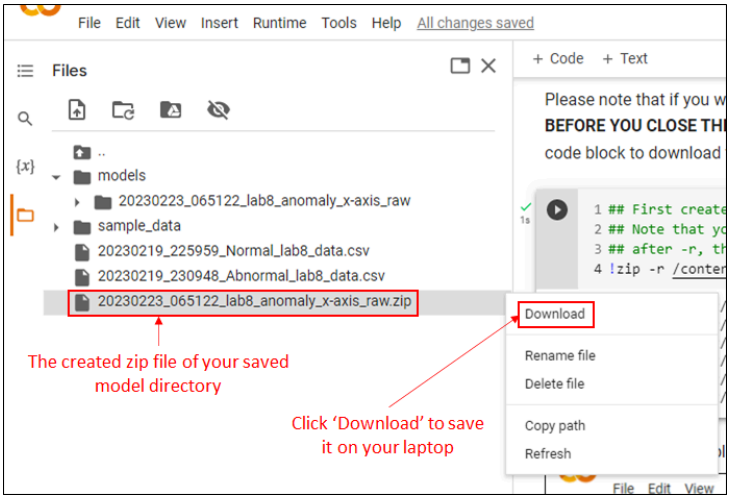

**If you run this Colab Notebook by mounting your Google Drive OR you run this on your laptop, you can skip this.**

Please note that if you want to reuse the saved model on different computer/platform, **YOU HAVE TO DOWNLOAD IT TO YOUR COMPUTER BEFORE YOU CLOSE THIS COLAB SESSION.** After you close/disconnect this Colab session, the saved model will be disappeared. Run the next code block to download the saved model as a zip file.

In [26]:
```
## First create a zip file for your saved model directory
## Note that your directory name must be different.
## after -r, the first arg. is the zip file, the second arg. is the target directory
!zip -r "/content/models/20260329_010240_lab8_anomaly_x-axis_raw.zip" \
     "/content/models/220260329_010240_lab8_anomaly_x-axis_raw"
```

```
zip warning: name not matched: /content/models/220260329_010240_lab8_anomaly_x-axis_raw

zip error: Nothing to do! (try: zip -r /content/models/20260329_010240_lab8_anomaly_x-axis_raw.zip . -i /cont
ent/models/220260329_010240_lab8_anomaly_x-axis_raw)
```

After you run the code block above, you see the created zip file from the saved model as the capture below. Try downloading it on your laptop.




Before we wrap up this lab, let's reload the saved model to see if it works.

In [27]:
```
model_path = "/content/models/20260329_212228_lab8_anomaly_x-axis_raw"

try:
    loaded = tf.saved_model.load(model_path)
    infer = loaded.signatures["serving_default"]

    # simple forward pass to verify functionality
    _ = infer(tf.zeros((1, N_feature)))

    print("Model reload success.")
except Exception as e:
```

```
    print("Model reload failed.")
    raise e
```

```
Model reload success.
```

## Task 2.3

After you perform reloading the saved model and run the code block above, the 'prediction' is a tuple that has two arrays. The first is boolean array and the second is float array. Explain these two arrays. For example, what do the elements in each array mean? Are they as expected?

---

The boolean array represents the models classification as normal or abnormal data based on the data. The float array is the model's confidence or normalized scores that this prediction of normal vs. abnormal categorization is correct. These are the expected behavior of a machine learning model.

---

## Task 2.4

Discuss how to improve the model.

---

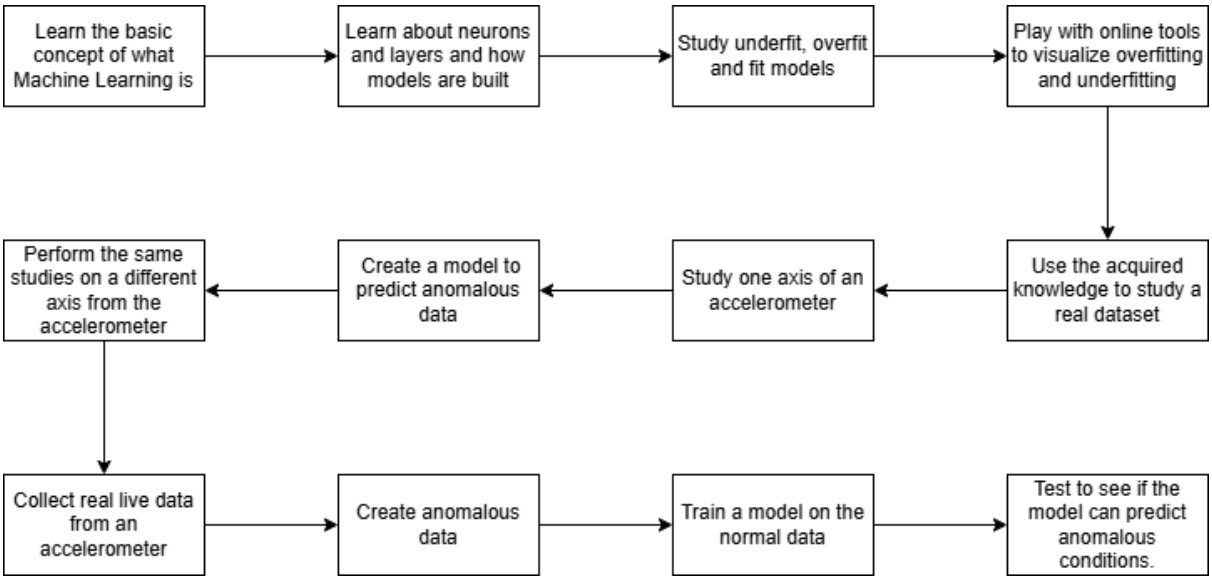
The model could be improved by using a different axis for the data. The model could also be improved by using a larger embedding layer and more epochs which would fit the data a bit more.

---

## Task 2.5

Model Unit 8 (prelab + lab) by creating a flow diagram that illustrates the logic, sequence, and processes of the unit.

---




---

# Lab8 Summary and Deliverables

## Deliverable 1 (Do this at home)

Using the given training Python code, find your best model for anomaly detection of the AFF. As a short report including figures, tables, equations, and so on, justify how and why the best model is selected. You have to include the following information. If you need, add code/text blocks.

- Hyper-parameters (Embedded size, batch size, epoch, etc.)
- Threshold of your model and how determined it
- What axis acceleration data is the best?
- Which feature among raw, time domain, and frequency domain is the best?
- Size of the feature
- What are the prediction performances (accuracy, precision, recall)?

In [28]:
```
### add and use code blocks
```

In [29]:
```
class DataSet():
  def __init__(self, axis, data_type="raw"):
    # Exploding the values contained in selected column and converting the string values into float values
    df_new = pd.concat([df['Condition'],df[axis].str.split(' ', expand=True).astype(float)], axis=1) # trans
    ds = df_new.copy() # make ds by copying df
    #Converting the Classifier into binary values
    ds.loc[df['Condition'] == 'Normal', 'Status'] = 1 # if Condition column is 'Normal', Give 'Status' Colum
    ds.loc[df['Condition'] == 'Abnormal', 'Status'] = 0 # if Condition column is 'Abnormal', Give 'Status' C
    ds.drop('Condition', axis=1, inplace=True) # drop 'Condition' column (the first column)
    data = ds.values

    # Define Raw data W/O signal processing
    raw_data = data[:,:-1]
    # Labels: The last column
    labels = data[:,-1]

    # get training and test (validation) dataset
    if data_type == "raw":
      input_feature = raw_data
    elif data_type == "time":
      input_feature = timeFeatures(raw_data) # define time domain feature
    elif data_type == "freq":
      input_feature = freqFeatures(raw_data) # define frequency domain feature (DFT)

    train_data, test_data, train_labels, test_labels = train_test_split(
        input_feature, labels, test_size=0.2, random_state=20
    )

    train_data = tensorNormalization(train_data) # Normalizing train data
    test_data = tensorNormalization(test_data) # Normalizing test data

    #Splitting the dataset based on classification: train_labels: Normal, ~train_labels: Abnormal
    train_labels = train_labels.astype(bool) # transform labels as bool (True or False)
    test_labels = test_labels.astype(bool) # in case of 'Normal' it is Ture, if not, False

    normal_train_data = train_data[train_labels] # define normal train data
    normal_test_data = test_data[test_labels] # define normal test data

    anomalous_train_data = train_data[~train_labels] # define abnormal train data
    anomalous_test_data = test_data[~test_labels] # define abnormal test data

    portion_of_anomaly_in_training = 0.1 #10% of training data will be anomalies
    end_size = int(len(normal_train_data)/(10-portion_of_anomaly_in_training*10))
    combined_train_data = np.append(normal_train_data, anomalous_test_data[:end_size], axis=0)

    # Class assignment
    self.normal_train_data = normal_train_data
    self.normal_test_data = normal_test_data
    self.anomalous_train_data = anomalous_train_data
    self.anomalous_test_data = anomalous_test_data
    self.train_data = train_data
    self.test_data = test_data
    self.train_labels = train_labels
    self.test_labels = test_labels
    self.N_feature = input_feature.shape[1] # Make N_feature an instance attribute
```

In [30]:
```
class AnomalyDetector(Model):
  def __init__(self, embedding_size, n_feature, **kwargs):
```

```
    super(AnomalyDetector, self).__init__(**kwargs)
    self.embedding_size = embedding_size
    self.n_feature = n_feature # Store n_feature as an attribute
    self.encoder = tf.keras.Sequential([
      layers.Dense(256, activation="relu"),
      layers.Dense(128, activation="relu"),
      layers.Dense(self.embedding_size, activation="relu")]) # Smallest Layer Defined Here

    self.decoder = tf.keras.Sequential([
      layers.Dense(128, activation="relu"),
      layers.Dense(256, activation="relu"),
      layers.Dense(self.n_feature, activation="sigmoid")]) # Use n_feature here

  def call(self, x):
    encoded = self.encoder(x)
    decoded = self.decoder(encoded)
    return decoded

  def get_config(self):
    config = super().get_config()
    config.update({
        "embedding_size": self.embedding_size,
        "n_feature": self.n_feature,
    })
    return config
```

In [31]:
```
from tensorflow.keras.callbacks import EarlyStopping
```

In [32]:
```
def find_best_model(
    embedding_sizes,
    learning_rates,
    batch_sizes,
    epochs_list,
    axes,
    data_types
):
    tf.random.set_seed(42)
    np.random.seed(42)

    best_roc_auc = 0
    best_model = None
    best_params = None

    results = []
    dataset_cache = {}

    for axis in axes:
        for data_type in data_types:
            # Process data better

            key = (axis, data_type)
            if key not in dataset_cache:
                dataset_cache[key] = DataSet(axis, data_type)
            data = dataset_cache[key]
            val_split = 0.5  # split normal_test into val + test-normal
            split_idx = int(len(data.normal_test_data) * val_split)

            normal_val_data = data.normal_test_data[:split_idx]
            normal_test_data = data.normal_test_data[split_idx:]

            # Final test set (balanced)
            test_data = np.concatenate([
                normal_test_data,
                data.anomalous_test_data
            ])

            test_labels = np.concatenate([
                np.zeros(len(normal_test_data)),
                np.ones(len(data.anomalous_test_data))
            ])
```

```
        for es in embedding_sizes:
            for lr in learning_rates:
                for bs in batch_sizes:
                    for ep in epochs_list:

                        tf.keras.backend.clear_session()

                        params = {
                            "Embedding Size": es,
                            "Learning Rate": lr,
                            "Batch Size": bs,
                            "Epoch": ep,
                            "Axis": axis,
                            "Data Type": data_type
                        }

                        print(params)
                        model = AnomalyDetector(es, data.N_feature)
                        model.compile(
                            optimizer=keras.optimizers.Adam(learning_rate=lr),
                            loss='mae'
                        )

                        # Early stopping to reduce time complexity
                        early_stop = EarlyStopping(
                            monitor='val_loss',
                            patience=5,
                            restore_best_weights=True
                        )

                        model.fit(
                            data.normal_train_data,
                            data.normal_train_data,
                            epochs=ep,
                            batch_size=bs,
                            validation_data=(normal_val_data, normal_val_data),
                            shuffle=True,
                            verbose=0,
                            callbacks=[early_stop]
                        )

                        # Model evaluation
                        recon = model.predict(test_data, verbose=0)
                        loss = tf.keras.losses.mae(recon, test_data).numpy()

                        fpr, tpr, _ = roc_curve(test_labels, loss)
                        roc_auc = auc(fpr, tpr)

                        print(f"ROC AUC: {roc_auc:.4f}")

                        results.append({
                            "params": params,
                            "roc_auc": roc_auc
                        })

                        if roc_auc > best_roc_auc:
                            print(f"New best: {roc_auc:.4f}")

                            best_roc_auc = roc_auc
                            best_model = model
                            best_params = params

                            model.save("best_model.h5")

    return best_model, best_params, results
```

## All Grid Search Parameters

The above code combines all of the data processing pipeline, model training, and model validation into a few short functions. Below shows a grid-search approach to optimize the model using the following options:

| | |
|---|---|
| Embedding Sizes | 16, 32, 64, 128, 256 |
| Learning Rates | 1e-1, 1e-2, 1e-3, 1e-4, 1e-5 |
| Batch Sizes | 16, 32, 64, 128, 256 |
| Epochs | 50, 100, 200, 300 |
| Axes | X, Y, Z |
| Data Types | Raw, Time Domain, Frequency Domain |

I used ChatGPT to optimize my code including postprocessing the data to also generate a validation set, and to include early stopping so that the number of epochs can be limited if additional epochs do not actually improve the model.

The function "find_best_model" iterates through these different combinations and greedily keeps the best parameters and models once they are trained and validated to provide the best possible model for the data.

In [100…

```
embedding_size = [16, 32, 64, 128, 256]
learning_rate = [1e-1, 1e-2, 1e-3, 1e-4, 1e-5]
batch_size = [16, 32, 64, 128, 256]
epoch = [50, 100, 200, 300]
axes = ["Xacc array [m/s2]", "Yacc array [m/s2]", "Zacc array [m/s2]"]
data_types = ["raw", "time", "freq"]

best_model, best_param, results = find_best_model(embedding_size, learning_rate, batch_size, epoch, axes, da
```

```
{'Embedding Size': 16, 'Learning Rate': 0.1, 'Batch Size': 16, 'Epoch': 50, 'Axis': 'Xacc array [m/s2]', 'Dat
a Type': 'raw'}
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)
`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.sav
e('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
ROC AUC: 0.9910
New best: 0.9910
Exception ignored in: <function WeakKeyDictionary.__init__.<locals>.remove at 0x795884b1afc0>
Traceback (most recent call last):
  File "/usr/lib/python3.12/weakref.py", line 369, in remove
    def remove(k, selfref=ref(self)):

KeyboardInterrupt:
{'Embedding Size': 16, 'Learning Rate': 0.1, 'Batch Size': 16, 'Epoch': 100, 'Axis': 'Xacc array [m/s2]', 'Da
ta Type': 'raw'}
ROC AUC: 0.9865
{'Embedding Size': 16, 'Learning Rate': 0.1, 'Batch Size': 16, 'Epoch': 200, 'Axis': 'Xacc array [m/s2]', 'Da
ta Type': 'raw'}
ROC AUC: 0.9865
{'Embedding Size': 16, 'Learning Rate': 0.1, 'Batch Size': 16, 'Epoch': 300, 'Axis': 'Xacc array [m/s2]', 'Da
ta Type': 'raw'}
ROC AUC: 0.9865
{'Embedding Size': 16, 'Learning Rate': 0.1, 'Batch Size': 32, 'Epoch': 50, 'Axis': 'Xacc array [m/s2]', 'Dat
a Type': 'raw'}
ROC AUC: 0.9865
{'Embedding Size': 16, 'Learning Rate': 0.1, 'Batch Size': 32, 'Epoch': 100, 'Axis': 'Xacc array [m/s2]', 'Da
ta Type': 'raw'}
ROC AUC: 0.9865
Exception ignored in: <function WeakKeyDictionary.__init__.<locals>.remove at 0x79588480cf40>
Traceback (most recent call last):
  File "/usr/lib/python3.12/weakref.py", line 369, in remove
    def remove(k, selfref=ref(self)):

KeyboardInterrupt:
```

{'Embedding Size': 16, 'Learning Rate': 0.1, 'Batch Size': 32, 'Epoch': 200, 'Axis': 'Xacc array [m/s2]', 'Data Type': 'raw'}
ROC AUC: 0.9865
{'Embedding Size': 16, 'Learning Rate': 0.1, 'Batch Size': 32, 'Epoch': 300, 'Axis': 'Xacc array [m/s2]', 'Data Type': 'raw'}

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_140683/366308029.py in <cell line: 0>()
      6 data_types = ["raw", "time", "freq"]
      7
----> 8 best_model, best_param, results = find_best_model(embedding_size, learning_rate, batch_size, epoch, a
xes, data_types)

/tmp/ipykernel_140683/1929009733.py in find_best_model(embedding_sizes, learning_rates, batch_sizes, epochs_l
ist, axes, data_types)
     84
     85                             # Model evaluation
---> 86                             recon = model.predict(test_data, verbose=0)
     87                             loss = tf.keras.losses.mae(recon, test_data).numpy()
     88

/usr/local/lib/python3.12/dist-packages/keras/src/utils/traceback_utils.py in error_handler(*args, **kwargs)
    115         filtered_tb = None
    116         try:
--> 117             return fn(*args, **kwargs)
    118         except Exception as e:
    119             filtered_tb = _process_traceback_frames(e.__traceback__)

/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/trainer.py in predict(self, x, batch_siz
e, verbose, steps, callbacks)
    586                 callbacks.on_predict_batch_begin(begin_step)
    587                 data = get_data(iterator)
--> 588                 batch_outputs = self.predict_function(data)
    589                 outputs = append_to_outputs(batch_outputs, outputs)
    590                 callbacks.on_predict_batch_end(

/usr/local/lib/python3.12/dist-packages/tensorflow/python/util/traceback_utils.py in error_handler(*args, **k
wargs)
    148     filtered_tb = None
    149     try:
--> 150       return fn(*args, **kwargs)
    151     except Exception as e:
    152       filtered_tb = _process_traceback_frames(e.__traceback__)

/usr/local/lib/python3.12/dist-packages/tensorflow/python/eager/polymorphic_function/polymorphic_function.py
in __call__(self, *args, **kwds)
    831
    832       with OptionalXlaContext(self._jit_compile):
--> 833         result = self._call(*args, **kwds)
    834
    835       new_tracing_count = self.experimental_get_tracing_count()

/usr/local/lib/python3.12/dist-packages/tensorflow/python/eager/polymorphic_function/polymorphic_function.py
in _call(self, *args, **kwds)
    917           )
    918       )
--> 919       return self._concrete_variable_creation_fn._call_flat(  # pylint: disable=protected-access
    920           filtered_flat_args,
    921           self._concrete_variable_creation_fn.captured_inputs,

/usr/local/lib/python3.12/dist-packages/tensorflow/python/eager/polymorphic_function/concrete_function.py in
_call_flat(self, tensor_inputs, captured_inputs)
   1320         and executing_eagerly):
   1321       # No tape is watching; skip to running the function.
-> 1322       return self._inference_function.call_preflattened(args)
   1323     forward_backward = self._select_forward_and_backward_functions(
   1324         args,

/usr/local/lib/python3.12/dist-packages/tensorflow/python/eager/polymorphic_function/atomic_function.py in ca
ll_preflattened(self, args)
    214   def call_preflattened(self, args: Sequence[core.Tensor]) -> Any:
    215     """Calls with flattened tensor inputs and returns the structured output."""
--> 216     flat_outputs = self.call_flat(*args)
    217     return self.function_type.pack_output(flat_outputs)
    218
```

```
/usr/local/lib/python3.12/dist-packages/tensorflow/python/eager/polymorphic_function/atomic_function.py in ca
ll_flat(self, *args)
    249         with record.stop_recording():
    250           if self._bound_context.executing_eagerly():
--> 251             outputs = self._bound_context.call_function(
    252                 self.name,
    253                 list(args),

/usr/local/lib/python3.12/dist-packages/tensorflow/python/eager/context.py in call_function(self, name, tenso
r_inputs, num_outputs)
   1686     cancellation_context = cancellation.context()
   1687     if cancellation_context is None:
-> 1688       outputs = execute.execute(
   1689           name.decode("utf-8"),
   1690           num_outputs=num_outputs,

/usr/local/lib/python3.12/dist-packages/tensorflow/python/eager/execute.py in quick_execute(op_name, num_outp
uts, inputs, attrs, ctx, name)
     51   try:
     52     ctx.ensure_initialized()
---> 53     tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle, device_name, op_name,
     54                                         inputs, attrs, num_outputs)
     55   except core._NotOkStatusException as e:

KeyboardInterrupt:
```

## Hyper Parameters

This section is a repeat of the code block above, but with fewer hyper parameters since not all combinations need to be tested after monitoring the training. If combinations are found that consistently perform higher than previous models, then only those specific characteristics need be used.

In [101…

```
embedding_size = [24]
learning_rate = [31e-3]
batch_size = [32]
epoch = [200]
axes = ["Yacc array [m/s2]"]
data_types = ["raw"]

best_model, best_param, results = find_best_model(embedding_size, learning_rate, batch_size, epoch, axes, da
```

```
{'Embedding Size': 24, 'Learning Rate': 0.031, 'Batch Size': 32, 'Epoch': 200, 'Axis': 'Yacc array [m/s2]',
'Data Type': 'raw'}
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)
`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.sav
e('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
ROC AUC: 0.7941
New best: 0.7941
```

## Validation Charts

Below are similar charts to before that show the performance of the model selected

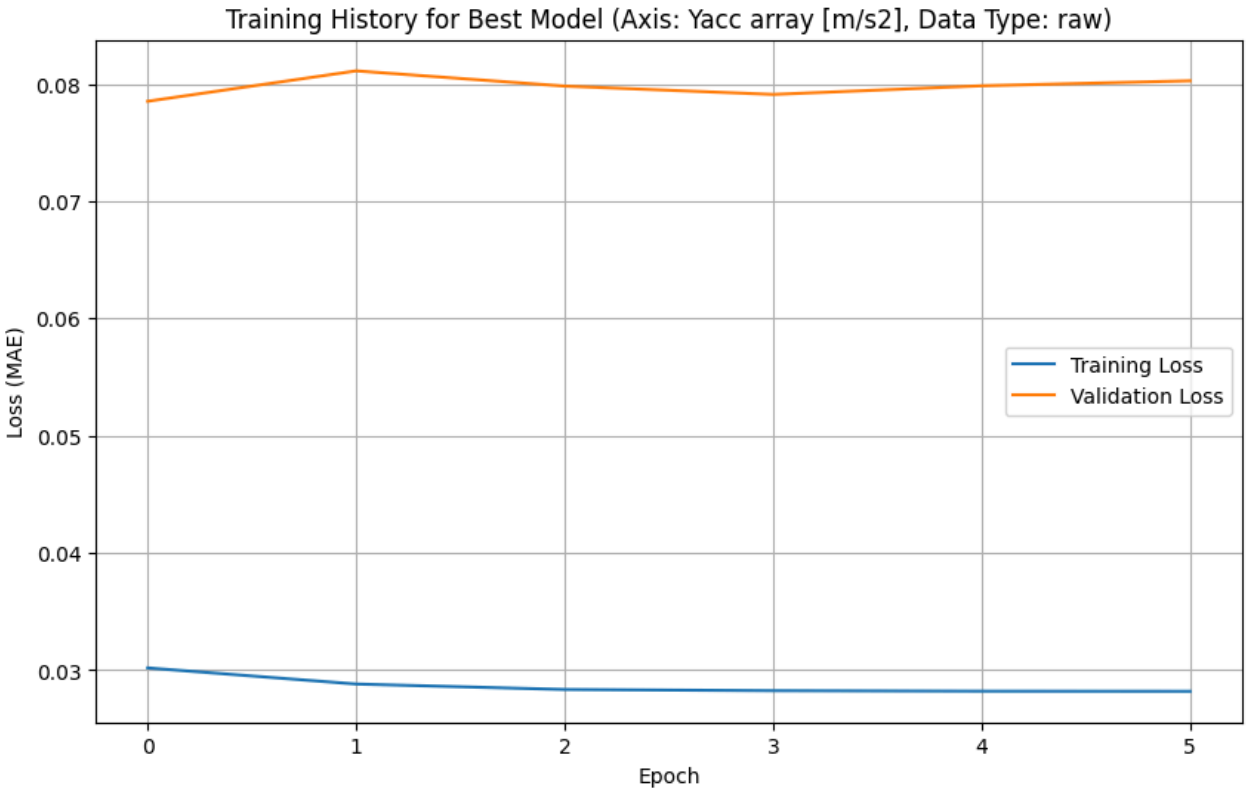
### Receiver Operating Characteristic

In [102…

```
# Recreate the DataSet for the best parameters
best_data = DataSet(best_param["Axis"], best_param["Data Type"])

# Recompile the best_model to prepare it for fitting
best_model.compile(optimizer=keras.optimizers.Adam(learning_rate=best_param["Learning Rate"]), loss='mae')

# Define early stopping as used during the original grid search
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
```

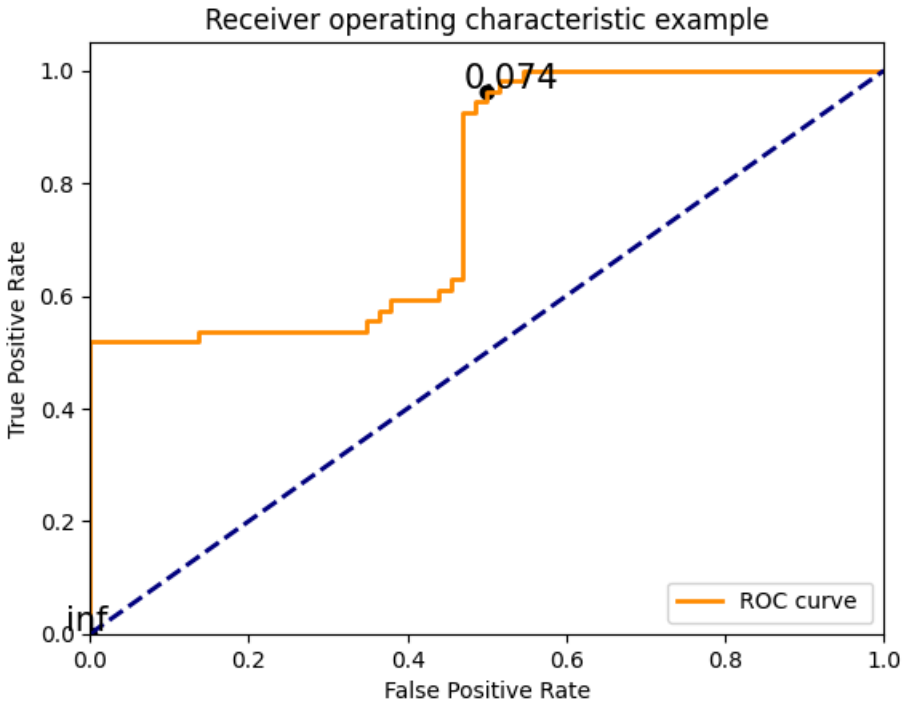
```
)

# Prepare validation data, as done in find_best_model
val_split = 0.5
split_idx = int(len(best_data.normal_test_data) * val_split)
normal_val_data = best_data.normal_test_data[:split_idx]

# Re-train the best model to capture its training history
history = best_model.fit(
    best_data.normal_train_data,
    best_data.normal_train_data,
    epochs=best_param["Epoch"],
    batch_size=best_param["Batch Size"],
    validation_data=(normal_val_data, normal_val_data),
    shuffle=True,
    verbose=1, # Change to 1 to see training progress
    callbacks=[early_stop]
)

# Plotting the evolution of training and validation loss
plt.figure(figsize=(10, 6))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title(f"Training History for Best Model (Axis: {best_param['Axis']}, Data Type: {best_param['Data Type']
plt.xlabel("Epoch")
plt.ylabel("Loss (MAE)")
plt.legend()
plt.grid(True)
plt.show()
```

```
Epoch 1/200
8/8 ━━━━━━━━━━━━━━━━━━━━ 3s 64ms/step - loss: 0.0302 - val_loss: 0.0785
Epoch 2/200
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0288 - val_loss: 0.0811
Epoch 3/200
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0284 - val_loss: 0.0798
Epoch 4/200
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0283 - val_loss: 0.0791
Epoch 5/200
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0282 - val_loss: 0.0798
Epoch 6/200
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0282 - val_loss: 0.0803
```




In [103…

```
best_data = DataSet(best_param["Axis"], best_param["Data Type"])
reconstructions = best_model(best_data.test_data)
loss = tf.keras.losses.mae(reconstructions, best_data.test_data)

fpr = []
tpr = []
#the test labels are flipped to match how the roc_curve function expects them.
# flipped_labels = 1-test_labels
flipped_labels = 1-test_labels
fpr, tpr, thresholds = roc_curve(flipped_labels, loss)
plt.figure()
lw = 2
plt.plot(fpr, tpr, color='darkorange',
         lw=lw, label='ROC curve ')
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic example')
plt.legend(loc="lower right")

# plot some thresholds
thresholds_every=20
thresholdsLength = len(thresholds)
colorMap=plt.get_cmap('jet', thresholdsLength)
for i in range(0, thresholdsLength, thresholds_every):
  threshold_value_with_max_four_decimals = str(thresholds[i])[:5]
  plt.scatter(fpr[i], tpr[i], c='black')
  plt.text(fpr[i] - 0.03, tpr[i] + 0.005, threshold_value_with_max_four_decimals, fontdict={'size': 15})

plt.show()
```



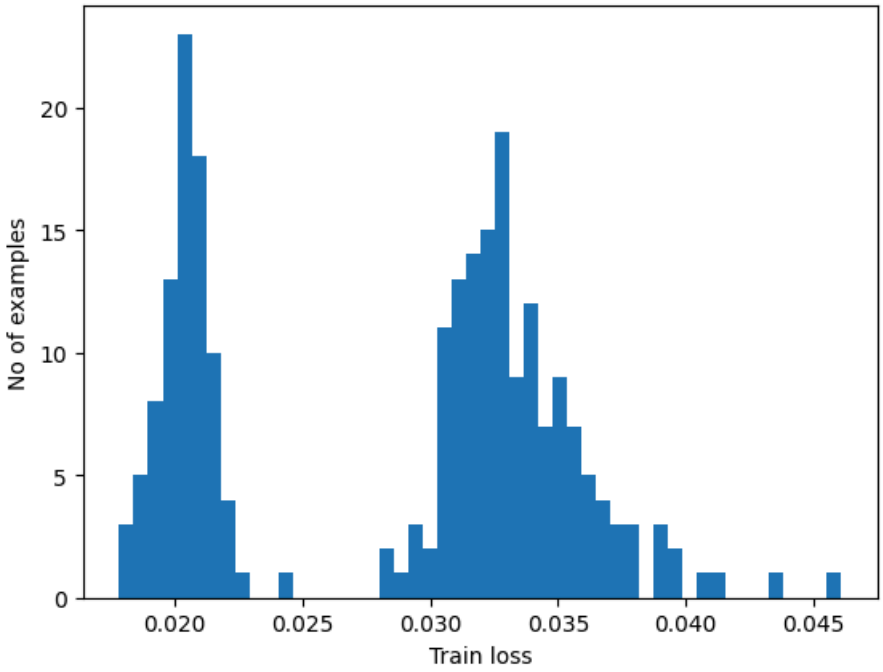

In [105…
```
roc_auc = auc(fpr, tpr) # check roc auc score (the area of roc graph above)
print("ROC - AUC score is {}.".format(roc_auc))
```

```
ROC - AUC score is 0.7836700336700337.
```

## Histogram of Training Model

In [104…
```
## reconstructions and histogram of training model
reconstructions = best_model.predict(best_data.normal_train_data)
train_loss = tf.keras.losses.mae(reconstructions, best_data.normal_train_data)

plt.hist(train_loss[None,:], bins=50)
plt.xlabel("Train loss")
plt.ylabel("No of examples")
plt.show()
```

```
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step
```



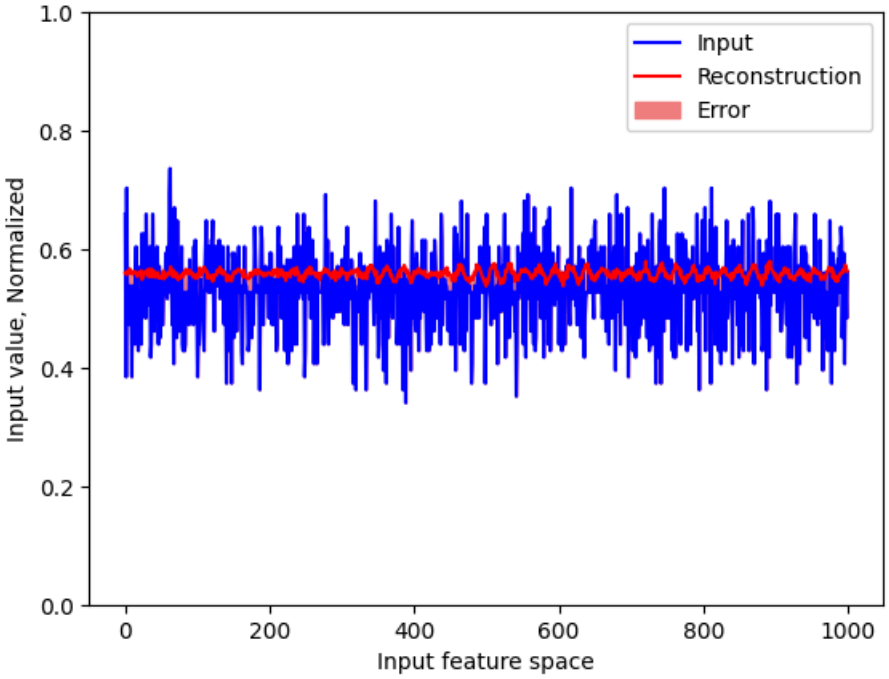

In [106…

```
# threshold =  #If you want to assign a value labeled in black in the ROC graph  <--------------------------
threshold = np.mean(train_loss) + np.std(train_loss) # Common rule of thumb based on the histogram of the tr
print("Threshold: ", threshold) # You have to note the threshold you will use
preds, scores = predict(best_model, best_data.test_data, threshold)
print_stats(preds, test_labels)
```

```
Threshold:  0.0354339
Accuracy = 0.55
Precision = 0.55
Recall = 1.0
```

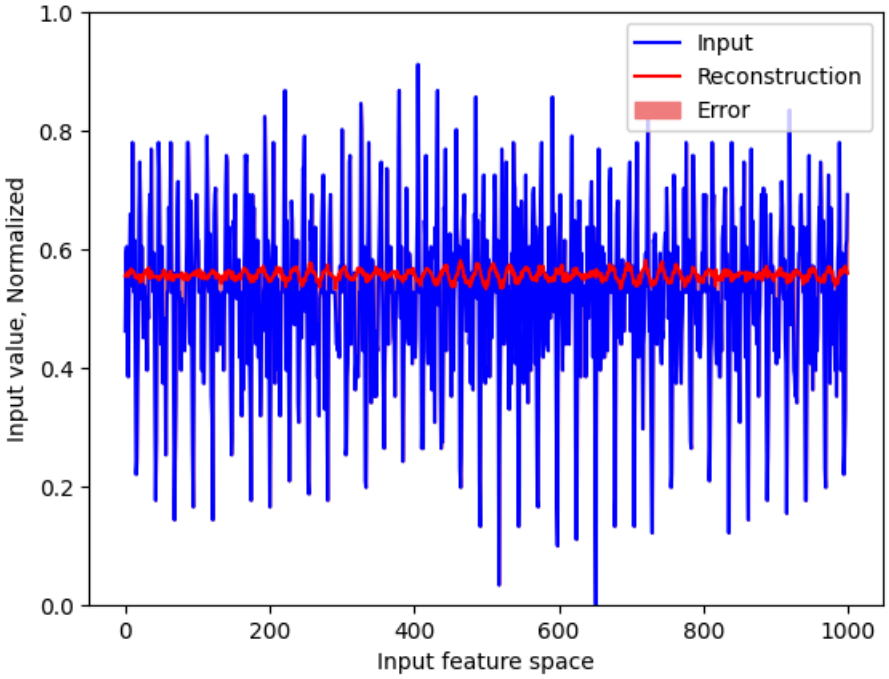
In [107…

```
# Plot "Normal" test data sample with reconstruction based on created model and error
encoded_data = autoencoder.encoder(normal_test_data).numpy()
decoded_data = autoencoder.decoder(encoded_data).numpy()

plt.plot(normal_test_data[0], 'b')
plt.plot(decoded_data[0], 'r')
plt.fill_between(np.arange(N_feature), decoded_data[0], normal_test_data[0], color='lightcoral')
plt.ylim([0, 1])
plt.legend(labels=["Input", "Reconstruction", "Error"])
plt.xlabel("Input feature space")
plt.ylabel("Input value, Normalized")
plt.show()
```




In [108…

```
# Plot "Abnormal" test data sample with reconstruction based on created model and error
encoded_data = autoencoder.encoder(anomalous_test_data).numpy()
decoded_data = autoencoder.decoder(encoded_data).numpy()

plt.plot(anomalous_test_data[0], 'b')
plt.plot(decoded_data[0], 'r')
plt.fill_between(np.arange(N_feature), decoded_data[0], anomalous_test_data[0], color='lightcoral')
plt.ylim([0, 1])
plt.legend(labels=["Input", "Reconstruction", "Error"])
plt.xlabel("Input feature space")
plt.ylabel("Input value, Normalized")
plt.show()
```




## Best Model Hyperparameters

After the grid search, the best model can be achieved using the following hyperparameters:

| | |
|---|---|
| Embedding Sizes | 24 |
| Learning Rates | 31e-3 |
| Batch Sizes | 32 |
| Epochs | 200 |
| Axis | Y |
| Data Types | Raw |

## Deliverable 2

Answer the following questions for your achievements

## Q1. Please summarize Lab8.

For Lab 8 we learned more about how machine learning works, how to develop a model and how to apply it to a real-world example. That of anomolous operating conditions of an air pump. This learning was then used for real data that was collected by using the muffin fan in lab. After data collection, a model was trained on normal operating condition data and then tested against both normal and anomolous data. These models were then ranked based on how accurate, precise and how much recall they have.

## Q2. What skills did you have to develop to accomplish this project?

To accomplish this project I had to learn a lot more about machine learning and how to go about training a model. My initial approach was very lengthy and after going to the drawing board and consulting ChatGPT I came up with a better way to automate the fine-tuning of an ML model.

## Q3. What aspects of this project were the most beneficial for your learning?

The aspects of the project that were the most beneficial for my learning were the visualizations of neural networks and being able to play with artificial data and smaller datasets before moving on to performing machine learning on much larger datasets.

## Q4. What challenges did you encounter in completing the project?

The main challenge I encountered while completing the project were that the grid search I initially developed took too long.

## Q5. How did you overcome the challenges or remedy the problems encountered?

After consulting with the TA, I learned that only a few hyperparameters needed to be compared to test and optimize the model. In addition, I did some research online and found a good benchmark rule of thumb ROC score to prevent my model from memorizing the data.

Get back to Lab Index Page